A Tour Of Sage

Version 10.10

The Sage Development Team

28 octobre 2026

Table des matières

1 La calculatrice Sage 3

2 Calcul numérique sous Sage 7

3 Les algorithmes inclus dans Sage 11

Cette courte présentation de Sage reprend le « Tour of Mathematica » proposé au début du « Mathematica Book ».

CHAPITRE 1

La calculatrice Sage

La ligne de commande Sage débute par `sage:`. Il ne vous est pas nécessaire de l'écrire à chaque ligne. Si vous utilisez le Notebook de Sage, vous n'avez qu'à recopier ce qui suit `sage:` dans une cellule, et à appuyer simultanément sur Maj + Entrée pour calculer le résultat.

```
sage: 3 + 5
8
```

```
>>> from sage.all import *
>>> Integer(3) + Integer(5)
8
```

Comme partout, l'accent circonflexe signifie « élever à la puissance ».

```
sage: 57.1 ^ 100
4.60904368661396e175
```

```
>>> from sage.all import *
>>> RealNumber('57.1') ** Integer(100)
4.60904368661396e175
```

Il permet de calculer des puissances d'objets plus complexes comme des matrices. Voici comment calculer l'inverse d'une matrice 2×2 avec Sage.

```
sage: matrix([[1,2], [3,4]])^(-1)
[  -2    1]
[ 3/2 -1/2]
```

```
>>> from sage.all import *
>>> matrix([[Integer(1),Integer(2)], [Integer(3),Integer(4)]])**(-Integer(1))
[  -2    1]
[ 3/2 -1/2]
```

Voici comment intégrer une fonction simple.

```
sage: x = var('x')   # Créer une variable symbolique
sage: integrate(sqrt(x)*sqrt(1+x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1) -␣
↪1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

```
>>> from sage.all import *
>>> x = var('x')   # Créer une variable symbolique
>>> integrate(sqrt(x)*sqrt(Integer(1)+x), x)
1/4*((x + 1)^(3/2)/x^(3/2) + sqrt(x + 1)/sqrt(x))/((x + 1)^2/x^2 - 2*(x + 1)/x + 1) -␣
↪1/8*log(sqrt(x + 1)/sqrt(x) + 1) + 1/8*log(sqrt(x + 1)/sqrt(x) - 1)
```

Les commandes suivantes permettent de demander à Sage de résoudre une équation quadratique. Le symbole == représente l'égalité sous Sage.

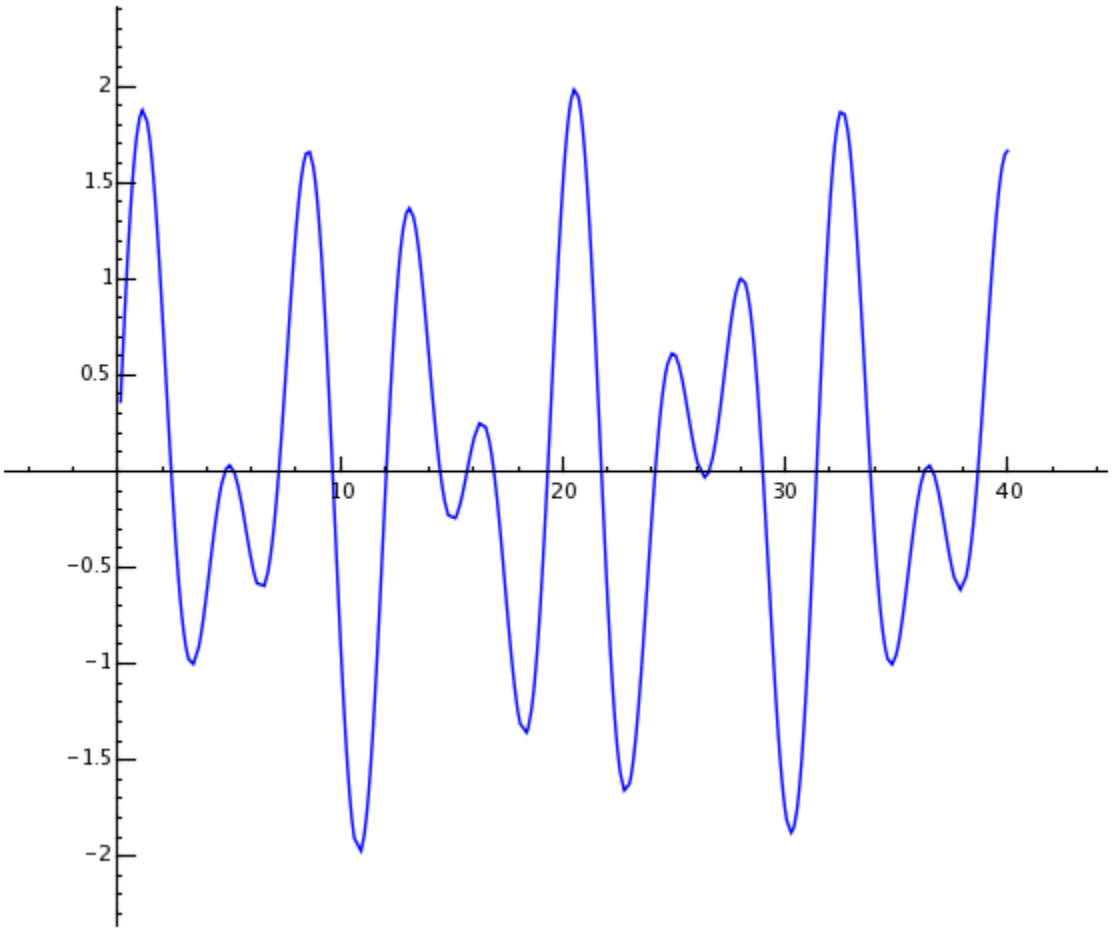
```
sage: a = var('a')
sage: S = solve(x^2 + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

```
>>> from sage.all import *
>>> a = var('a')
>>> S = solve(x**Integer(2) + x == a, x); S
[x == -1/2*sqrt(4*a + 1) - 1/2, x == 1/2*sqrt(4*a + 1) - 1/2]
```

Le résultat est une liste d'inégalités.

```
sage: S[0].rhs()
-1/2*sqrt(4*a + 1) - 1/2
sage: show(plot(sin(x) + sin(1.6*x), 0, 40))
```

```
>>> from sage.all import *
>>> S[Integer(0)].rhs()
-1/2*sqrt(4*a + 1) - 1/2
>>> show(plot(sin(x) + sin(RealNumber('1.6')*x), Integer(0), Integer(40)))
```


2
1.5
1
0.5
−0.5
−1
−1.5
−2
10
20
30
40

CHAPITRE 2

Calcul numérique sous Sage

Tout d'abord, créons une matrice aléatoire de taille 500×500.

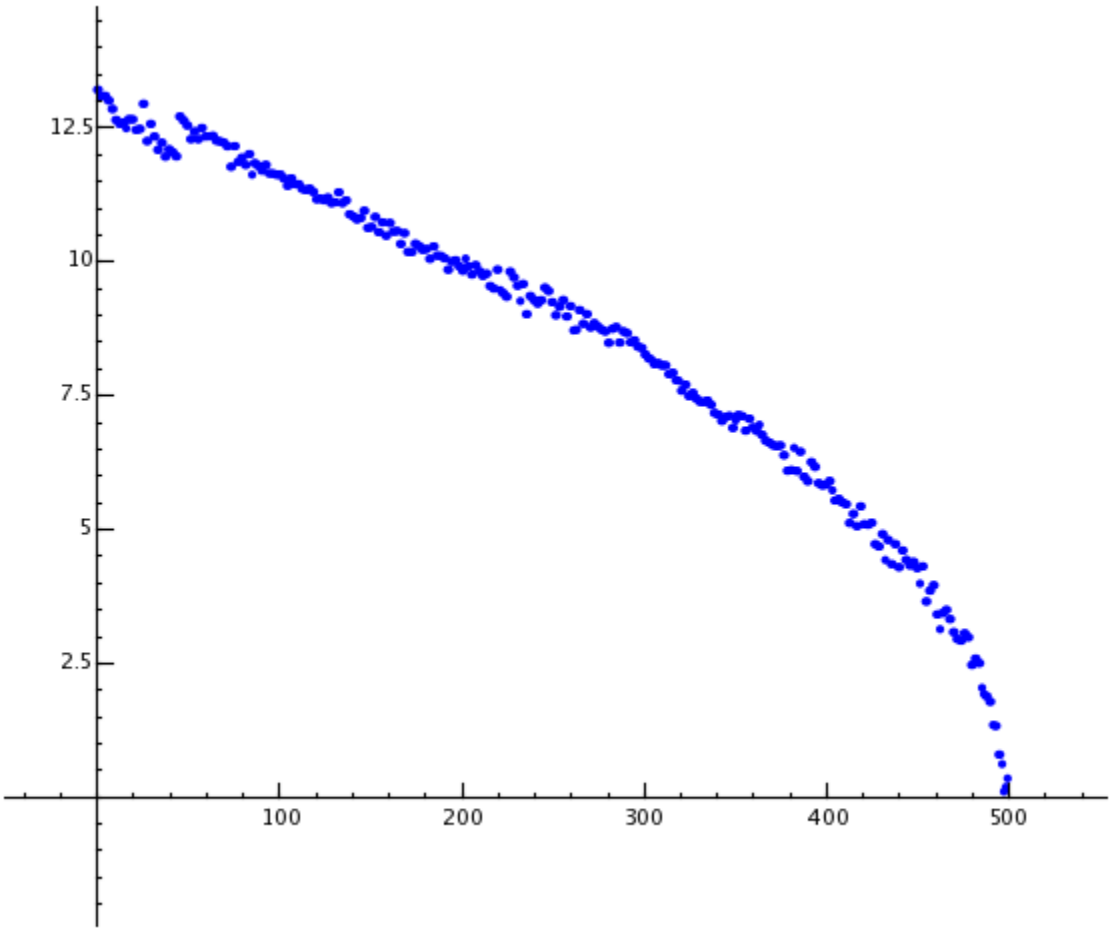
```
sage: m = random_matrix(RDF,500)
```

```
>>> from sage.all import *
>>> m = random_matrix(RDF,Integer(500))
```

Il ne faut que quelques secondes à Sage pour calculer les valeurs propres de la matrice et en faire un graphique.

```
sage: e = m.eigenvalues()  # environ 2 secondes
sage: w = [(i, abs(e[i])) for i in range(len(e))]
sage: show(points(w))
```

```
>>> from sage.all import *
>>> e = m.eigenvalues()  # environ 2 secondes
>>> w = [(i, abs(e[i])) for i in range(len(e))]
>>> show(points(w))
```



Grâce à la bibliothèque GMP (GNU Multiprecision Library), Sage peut effectuer des calculs sur de très grands nombres, comportant des millions ou des milliards de chiffres.

```
sage: factorial(100)
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
sage: n = factorial(1000000)  # environ 2.5 secondes
```

```
>>> from sage.all import *
>>> factorial(Integer(100))
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565182862536979208272237582511852109168640000000000000000000000000
>>> n = factorial(Integer(1000000))  # environ 2.5 secondes
```

Voici comment afficher les 100 premières décimales de π.

```
sage: N(pi, digits=100)
3.1415926535897932384626433832795028841971693993751058209749445923078164062862089986280348253421170
```

```
>>> from sage.all import *
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> N(pi, digits=Integer(100))
3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862
↪034825342117068
```

Voici comment Sage factorise un polynôme en deux variables.

```
sage: R.<x,y> = QQ[]
sage: F = factor(x^99 + y^99)
sage: F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
sage: F.expand()
x^99 + y^99
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> F = factor(x**Integer(99) + y**Integer(99))
>>> F
(x + y) * (x^2 - x*y + y^2) * (x^6 - x^3*y^3 + y^6) *
(x^10 - x^9*y + x^8*y^2 - x^7*y^3 + x^6*y^4 - x^5*y^5 +
 x^4*y^6 - x^3*y^7 + x^2*y^8 - x*y^9 + y^10) *
(x^20 + x^19*y - x^17*y^3 - x^16*y^4 + x^14*y^6 + x^13*y^7 -
 x^11*y^9 - x^10*y^10 - x^9*y^11 + x^7*y^13 + x^6*y^14 -
 x^4*y^16 - x^3*y^17 + x*y^19 + y^20) * (x^60 + x^57*y^3 -
 x^51*y^9 - x^48*y^12 + x^42*y^18 + x^39*y^21 - x^33*y^27 -
 x^30*y^30 - x^27*y^33 + x^21*y^39 + x^18*y^42 - x^12*y^48 -
 x^9*y^51 + x^3*y^57 + y^60)
>>> F.expand()
x^99 + y^99
```

Il ne faut pas plus de 5 secondes à Sage pour calculer le nombre de façons de partitionner mille millions (10^8) comme une somme d'entiers positifs.

```
sage: z = Partitions(10^8).cardinality()  # environ 4.5 secondes
sage: str(z)[:40]
'1760517045946249141360373894679135204009'
```

```
>>> from sage.all import *
>>> z = Partitions(Integer(10)**Integer(8)).cardinality()  # environ 4.5 secondes
>>> str(z)[:Integer(40)]
'1760517045946249141360373894679135204009'
```

CHAPITRE 3

Les algorithmes inclus dans Sage

Quand vous utilisez Sage, vous avez accès à l'une des plus grandes collections Open Source d'algorithmes de calcul.